

Lab Assignment No.	06
Title	Perform Sentiment Analysis in the network graph using RNN.
Roll No.	
Class	BE
Date of Completion	
Subject	Computer Laboratory-IV: Deep Learning
Assessor's Sign	

ASSIGNMENT No: 06

Title:- Perform sentiment analysis with a recurrent neural networks RNN

Objective: Implement a Recurrent Neural Network (RNN) on a network graph for sentiment analysis.

Outcome: Solve the language translation problem by Recurrent neural network(RNN)

Software Requirements:

- Python (3.x recommended)
- Jupyter Notebook or any Python IDE

Hardware Requirements:

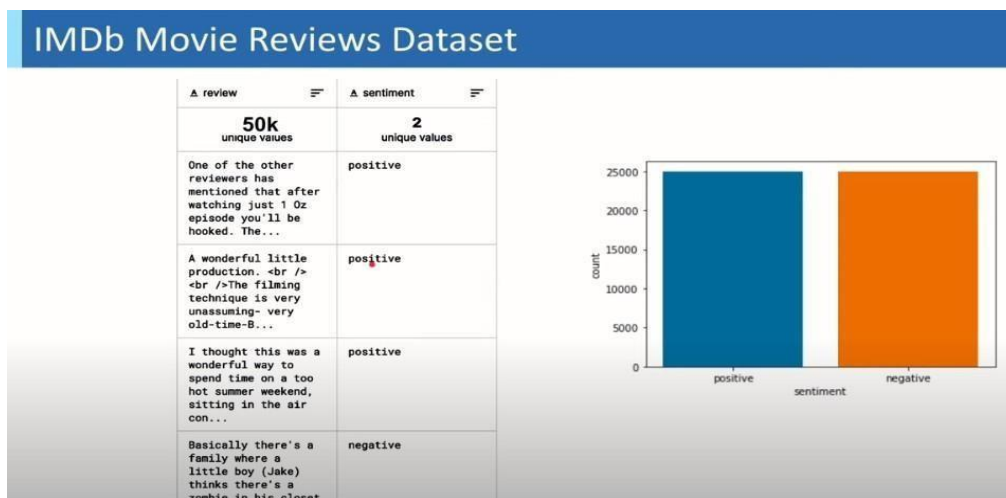
- A machine with sufficient RAM and processing power for model training (8GB RAM recommended)

Prerequisites:

- Basic understanding of Python programming
- Familiarity with the concepts of Neural Networks

Dataset:

https://github.com/skillcate/sentiment-analysis-with-deep-neural-networks/blob/main/a1_IMDB_Dataset.csv

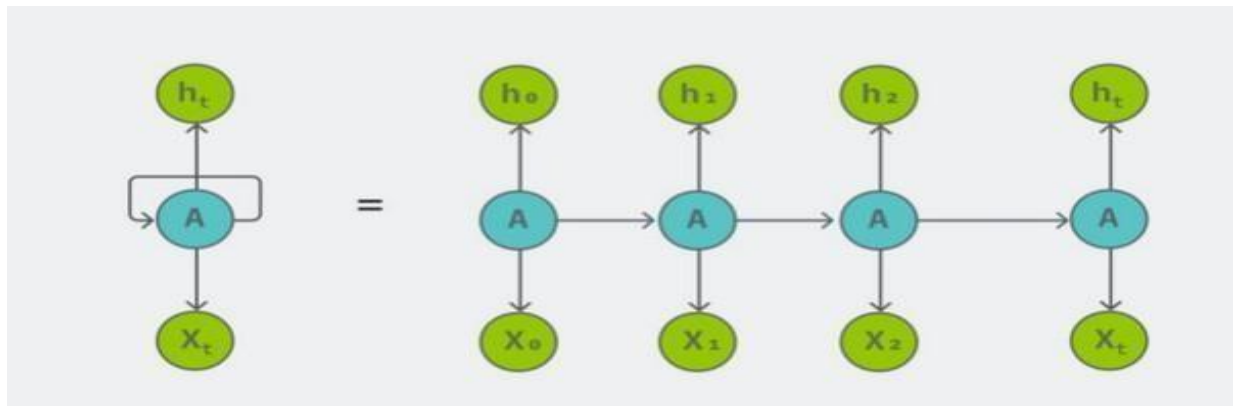


Module Used:

- TensorFlow or PyTorch
- NumPy
- Matplotlib

Theory:

A Recurrent Neural Network (RNN) represents a sophisticated extension of the traditional feed forward neural network by incorporating an internal memory mechanism. In contrast to feed forward networks, RNNs possess a recurrent nature, executing the same operation for each input data while considering the outcome of the preceding computation. This iterative process involves generating an output, which is then replicated and fed back into the recurrent network. In making decisions, the RNN takes into account both the current input and the knowledge acquired from the previous input.



RNNs, uniquely suited for tasks like unsegmented connected handwriting recognition or speech recognition, leverage their internal state (memory) to effectively process sequences of inputs. Unlike other neural networks where inputs are treated independently, RNNs establish a relationship among all inputs in a sequence.

The sequence begins with the initial input, $x(0)$, producing an output, $h(0)$, which, together with $x(1)$, becomes the input for the subsequent step. This process continues iteratively, with each hidden state (h) from the previous step being combined with the current input (x) for the next computation.

This sequential progression ensures that the network retains contextual information throughout the training process.

The formula for the current state involves weight matrices (W) for the current input (W_{hx}) and the previous hidden state (W_{hh}), alongside an activation function, typically $\tanh$. This activation function introduces non-linearity, compressing activations within the range of $[-1, 1]$. The final output state (Y_t) is determined by the weight matrix (W_{hy}) at the output layer.

$$h_t = f(h_{t-1}, x_t)$$

Applying Activation Function:

$$h_t = \tanh (W_{hh}h_{t-1} + W_{hx}x_t)$$

$$y_t = W_{hy}h_t$$

Y_t is the output state. W_{hy} is the weight at the output state.

In essence, RNNs excel at learning and remembering dependencies between inputs across sequential data, making them invaluable for tasks that involve context and temporal relationships.

Algorithm:

Applying Recurrent Neural Networks (RNNs) to network graphs for sentiment analysis involves a step-by-step process. Here's a high-level algorithm:

1. Data Preparation:

- Collect a dataset with network graph data and sentiment labels for each node or edge.
- Represent the graph as an adjacency matrix or an edge list.
- Assign sentiment labels to nodes or edges in the graph.

2. Node/Edge Embedding:

- Represent each node or edge in the graph using embeddings. You can use techniques like Word2Vec, GloVe, or Graph Embeddings (e.g., GraphSAGE) to convert nodes into fixed-size vectors.

3. Sequence Generation:

- Create sequences of node or edge embeddings for each path or subgraph in the network. The sequence length can vary based on the application and context.

4. Labeling:

- Assign sentiment labels to the sequences based on the sentiment of the nodes or edges in the sequence. This is your target variable for training the RNN.

5. RNN Model Architecture:

- Choose an RNN architecture suitable for sequence processing. Long Short-term Memory (LSTM) or Gated Recurrent Unit (GRU) cells are often used due to their ability to capture long- range dependencies.
- Define the input layer to accept sequences of node or edge embeddings.
- Optionally, include additional layers like Bidirectional RNNs or attention mechanisms for improved performance.

6. Training:

- Split the dataset into training, validation, and test sets.
- Train the RNN model on the training set using backpropagation through time (BPTT) or other sequence training methods.
- Optimize the model using an appropriate loss function (e.g., categorical cross-entropy) and an optimizer (e.g., Adam).

7. Evaluation:

- Evaluate the trained model on the validation set to tune hyperparameters and prevent overfitting.
- Test the model on the test set to assess its performance on unseen data.

8. Inference:

- Use the trained RNN model for sentiment analysis on new or unseen network graphs.

9. Post-Processing:

- Analyze the results, and if needed, perform post-processing or fine-tuning to improve the model's accuracy.

10. Visualization (Optional):

- Visualize the sentiment predictions on the network graph to gain insights into sentiment patterns and relationships.

Let's understand using an example:

Let's break down the algorithm with a simplified example of sentiment analysis on a network graph. In this example, we'll consider a social network where nodes represent users, and edges represent connections between users. The task is to predict the sentiment (positive or negative) of interactions between users.

Example:

1. Data Preparation:

- Collect a dataset with a social network graph, where each node represents a user and each edge represents a connection.
- Assign sentiment labels (positive or negative) to edges based on the sentiment of interactions between users.

2. Node Embedding:

- Represent each user (node) with Word2Vec embeddings based on their interactions or posts in the social network.

3. Sequence Generation:

- Create sequences of node embeddings for each path of interactions between users. For example, if we have the following interactions: A -> B (positive), B -> C (negative), C -> D (positive), the sequence for user A could be [A_embedding, B_embedding].

4. Labeling:

- Assign sentiment labels to the sequences. For instance, if the sequence is [A_embedding, B_embedding] and the sentiment of the interaction B -> C is negative, then the label for this sequence is negative.

5. RNN Model Architecture:

- Use an LSTM-based RNN architecture for sequence processing.
- Define the input layer to accept sequences of user embeddings.
- Add LSTM layers to capture the temporal dependencies in the sequence.

6. Training:

- Split the dataset into training, validation, and test sets.
- Train the RNN model on the training set using backpropagation through time (BPTT).
- Optimize the model using categorical cross-entropy as the loss function and the Adam optimizer.

7.Evaluation:

- Evaluate the trained model on the validation set to tune hyperparameters and prevent overfitting.
- Test the model on the test set to assess its performance on unseen interactions.

8.Inference:

- Use the trained RNN model for sentiment analysis on new interactions between users.

9. Post-Processing:

- Analyze the results, and if needed, perform post-processing or fine-tuning to improve the model's accuracy.

10.Visualization (Optional):

- Visualize the predicted sentiments on the social network graph to understand sentiment patterns and relationships between users.

This example outlines a simplified algorithm for sentiment analysis on a social network graph using an RNN. Keep in mind that real-world scenarios may involve more complex data preprocessing, model tuning, and feature engineering based on the characteristics of your dataset

Application:

1. Social Media Sentiment Analysis:

- Analyzing sentiments in interactions between users on social media platforms.
- Identifying positive or negative sentiments in comments, replies, and discussions.

2. Online Community Monitoring:

- Monitoring sentiments within online communities or forums.
- Identifying potential issues, conflicts, or positive interactions within community discussions.

3. Brand Reputation Management:

- Analyzing sentiments related to a brand by examining interactions between users mentioning the brand.
- Monitoring sentiment trends over time and identifying areas for improvement.

4. Customer Feedback Analysis:

- Analyzing sentiments in customer reviews and feedback.
- Understanding the overall sentiment regarding a product or service.

5. Collaborative Filtering in Recommendation Systems:

- Utilizing sentiments in user-item interactions to improve collaborative filtering recommendation systems.
- Personalizing recommendations based on both historical interactions and sentiments.

6. Financial News Analysis:

- Analyzing sentiments in financial news articles and discussions between investors.
- Understanding market sentiment and potential impacts on stock prices.

7. Healthcare Interactions:

- Analyzing sentiments in interactions between patients and healthcare professionals.
- Identifying trends in patient satisfaction or areas for improvement in healthcare services.

8. Educational Platforms:

- Analyzing sentiments in interactions between students and instructors on educational platforms.
- Monitoring student engagement and identifying potential issues or areas for improvement.

9. Political Discourse Analysis:

- Analyzing sentiments in interactions between individuals discussing political topics.
- Identifying trends in public opinion and potential areas of concern.

10. Influencer Marketing:

- Analyzing sentiments in interactions between influencers and their followers.
- Understanding the impact of influencer content on audience sentiment.

These applications highlight the versatility of sentiment analysis using RNNs on network graphs, where the sequential nature of interactions and relationships plays a crucial role in understanding sentiments in various domains.

Conclusion: Thus Performed Sentiment Analysis in the network graph using RNN.